ENTREGA

- El nombre del fichero será 401.zip y contendrá:
  - `mongodb.js`: archivo que contiene el código para generar la base de datos MongoDB .
  - `basex.xml`: archivo XML para generar la base de datos en BaseX.
  - El código de la aplicación. Solo será necesario la carpeta `src`, es decir, el directorio raíz de los paquetes con los archivos `.java`.

A TENER EN CUENTA

Se debe **crear un menú** que permita identificar, de forma clara y concisa, cada una de las acciones que se piden implementar. Además, **todos los mensajes** que se muestren, ya sean de información, de introducción de datos o de error deberán, también, ser **claros y concisos**.

Cualquier código que:

- Se detecte que es una copia
- No compile ni ejecute

No será corregido y su puntuación será de `0`. Independientemente de lo que se haya hecho.

Se deberá intentar obtener el resultado directamente con una consulta en XQuery o en MongoDB. **Utilizar Java para obtener el resultado cuando existe una posible consulta será penalizado**.

Descripción del proyecto

Se pide desarrollar una plataforma de comercio electrónico que permita a los usuarios explorar productos, realizar pedidos y gestionar sus cuentas. La aplicación combinará el uso de dos bases de datos:

• **Una base de datos estática con datos XML** (en BaseX), llamada **productos**, para almacenar información estática sobre los productos.

EJEMPLO DE XML

Aquí podéis descargar un ejemplo válido de fichero XML que se añadirá en la base de datos BaseX.

• **Una base de datos MongoDB,** llamada **tienda** para gestionar detalles dinámicos como pedidos, clientes y carritos de compras.

POSIBLE ESTRUCTURA DE LA BASE DE DATOS MONGO

A modo de ayuda y posible idea, sería conveniente crear tres colecciones:

- `Clientes`: almacena la información personal de cada cliente.
- `Carrito`: contiene la lista de productos que el usuario está mirando de comprar.
- `Pedidos`: son los productos que el usuario ha comprado.

Aquí tenéis un ejemplo de posible estructura. Aunque lo haya puesto en un solo fichero es recomendable gestionarlo en tres colecciones independientes aun así, cada uno puede implementarlo como prefiera.

Echadle un vistazo a la sección sobre las estructuras de los documentos en donde se habla de las diferentes formas de enlazar colecciones en MongoDB.

Tareas a realizar

Se pide implementar un menú que permita realizar las siguientes operaciones sobre la base de datos BaseX y MongoDB. Una posible estructura de menú puede ser la siguiente.

```
CONSULTAS BASE DE DATOS XML
    1. DESCRIPCIÓN DE LA OPERACIÓN 1
    2. DESCRIPCIÓN DE LA OPERACIÓN 2.
    ...
    7. DESCRIPCIÓN DE LA OPERACIÓN 7.

CONSULTAS BASE DE DATOS MONGODB
    8. DESCRIPCIÓN DE LA OPERACIÓN 8
    ...
    17. DESCRIPCIÓN DE LA OPERACIÓN 17
```

En cuanto a las operaciones que se deberán realizar, serán las siguientes:

Sobre la base de datos XML se pide:

1. Modificar el valor de un elemento de un XML según un ID.

2. Eliminar un **producto** según su ID.

3. Consulta 1: Obtener todos los productos por orden alfabético del nombre (se mostrarán los siguientes campos: i**d, nombre, precio, disponibilidad** y **categoria**).

4. Consulta 2: Listar productos con una disponibilidad mayor a X unidades (se mostrarán los siguientes campos: i**d, nombre, precio, disponibilidad** y c**ategoria**).

5. Consulta 3: Mostrar la **categoría**, el **nombre** y el **precio** del producto más caro para cada **categoría**. En el caso de haber varios se devolverá el de la primera posición.

6. Consulta 4: Mostrar el **nombre** de los productos y su **fabricante** para aquellos productos cuya **descripción** incluya una subcadena. Se deberá mostrar la información ordenada según el nombre del **fabricante** de forma inversa al alfabeto.

7. Consulta 5: Mostrar la cantidad total de productos en cada categoría (teniendo en cuenta el elemento **disponibilidad)** y calcular el porcentaje que representa respecto al total de productos.

> ⚠ **NOTA SOBRE LAS CONSULTAS ANTERIORES**
>
> Todas las consultas anteriores tienen una consulta XQuery que permite obtener el resultado de forma directa

Sobre la base de datos MongoDB se pide:

8. Crear un nuevo cliente (no podrá haber **email** repetidos).

9. Identificar cliente según el **email**. Dado el email se obtendrá el **ID del cliente** de forma que las siguientes consultas se harán sobre ese cliente. Para cambiar de cliente se tendrá que volver a seleccionar esta opción.

10. Borrar un cliente.

11. Modificar el valor de un campo de la información del cliente.

12. Añadir producto al carrito del cliente. Se pedirá: **id del producto** y **cantidad**, así como si se desea seguir introduciendo más productos.

13. Mostrar el carrito del cliente. Se mostrarán los datos del carrito y el precio total.

14. Mostrar pedidos del cliente.

15. Pagar el carrito de un cliente: se mostrará el carrito junto con una orden de confirmación. Si la orden es positiva se pasarán todos los productos a formar parte de un nuevo pedido.

16. Consulta 1: Teniendo en cuenta todos los clientes, calcular el total de la compra para cada carrito y listar los resultados ordenados por el total de forma ascendente. (No es necesario tener en cuenta la multiplicación de **precio_unitario * cantidad** con sumar los **precio_unitario** es suficiente).

17. Consulta 2: Teniendo en cuenta todos los clientes, obtener el total gastado por cada cliente en todos sus pedidos.

> ⚠ **NOTA SOBRE LOS APARTADOS ANTERIORES**
>
> Para la realización de los **apartados 14, 16 y 17**:
>
> - Si se ha diseñado la BD con una única colección existe una consulta en MongoDB que permite obtener el resultado de forma directa.
> - Si se ha diseñado la BD utilizando varias colecciones se puede usar Java para obtener los datos de una colección para añadirlos a la consulta de la otra colección. Pero no para formatear o calcular los datos finales.